CONFIDENTIAL

Faculty of Computing

UNIVERSITI PENYELIDIKAN

UNIVERSITI TEKNOLOGI MALAYSIA

TEST 1 (2022/2023-2)

SUBJECT CODE	**:**	**SECJ2154**
SUBJECT NAME	**:**	**OBJECT ORIENTED PROGRAMMING**
YEAR/PROGRAM	**:**	**2 SECB / SECJ / SECP / SECR / SECV**
DURATION	**:**	**2 HOURS**
DATE/TIME	**:**	**17 May 2023 / 8:00 PM – 10:00 PM (MYT)**
VENUE	**:**	**N28 (BK1 – BK7)**

INSTRUCTIONS:

THIS QUESTION BOOK CONSISTS OF 2 SECTIONS:

SECTION A: 15 Objective Questions 30 Marks

SECTION B: 3 Structured Questions 70 Marks

ANSWER ALL QUESTIONS IN THE ANSWER BOOKLET PROVIDED
THE QUESTION BOOKLET MUST BE RETURNED WITH THE ANSWER BOOKLET

(Write your lecturer's name and section in the question and answer booklets)

Name	
IC	
Year/Program	
Section	
Lecturer Name	

This question book consists of **13** (**THIRTEEN**) printed pages excluding this page.

SECTION A: OBJECTIVE QUESTIONS (30 Marks)

Section A consists of 15 objective questions. Choose the correct answer, and write your answer in the answer booklet. Each question carries 2 marks.

1. What is the object-oriented programming (OOP) feature that allows methods and attributes to be combined into a single entity called?

   A. Data hiding
   B. Encapsulation
   C. Polymorphism
   D. Inheritance

2. Which of the following OOP principles is used to hide data from other objects in the program?

   A. Data hiding
   B. Encapsulation
   C. Polymorphism
   D. Abstraction

3. Classes can be used to derive or instantiate many _____

   A. Attributes
   B. Objects
   C. Methods
   D. Objects and methods

4. Which of the characteristics of OOP have the concept of creating a new class from an existing class?

   A. Abstraction
   B. Polymorphism
   C. Encapsulation
   D. Inheritance

5. Which one of the following is related with code reusability in OOP?

   A. Abstraction
   B. Encapsulation
   C. Inheritance
   D. Polymorphism

6. Which one of the following is a class element that holds data?

   A. Attribute
   B. Constructor
   C. Method
   D. Class

7. What elements can be used to set the value of class member data?

   A. Class attribute
   B. Constructor
   C. Method
   D. Constructor and method

8. The methods that retrieve class member data are called _____

   A. Accessors
   B. Mutators
   C. Constructors
   D. Setters

9. Which modifier allows other classes to access data, methods, or classes?

   A. Final modifier
   B. Private Modifier
   C. Protected modifier
   D. Public modifier

10. Local variables are variables declared within _____

    A. Class
    B. Constructor
    C. Method
    D. Package

11. Which of these is a mechanism for naming and visibility control of a class and its content?

    A. Object
    B. Packages
    C. Interfaces
    D. References

12. What are the two types of variables commonly used in Java programs?

A. Primitive and primary variables
B. Indexing and referencing variables
C. Static and local variable
D. Primitive and reference variables

13. A variable declared inside the class but outside the body of the method is called _____

A. An instance variable
B. A local variable
C. Static variable
D. External variable

14. Which of the following is not true about constructor?

A. Constructor is called during the instantiation of object
B. A single class can have many constructors
C. Constructor can change member data after instantiation of object
D. Value of class member data can be initialized inside the constructor

15. Consider the following class diagram:

Person
name : string age : int
getAge() : int

Which of the following is best identified as behavior of the class?

A. Person
B. name
C. age
D. getAge()

SECTION B: STRUCTURED QUESTIONS (70 Marks)

Section B consists of 3 structured questions. Answer all questions and write your answer in the answer booklet.

Question 1 (20 Marks)

1.1 An UML class diagram of a `Loan` class has been prepared as follows:

```
Loan
-------------------------------------
- INTEREST=7.2 : double
- type : String
- years : int
- amount : double
-------------------------------------
+Loan(String, int, double)
+setType(String): void
+setYear(int) : void
+setAmount(double) : void
+getType(): String
+getYears() : int
+getAmount() : double
```

Write a Java code to implement `Loan` class as represented by the UML class diagram above.

(13 marks)

1.2 Below is a Java program (`TestLoan.java`) to test the `Loan` and `LoanManager` classes:

```
// TestLoan.java
public class TestLoan {
   public static void main(String[] args) {
      System.out.println("Test Loan and LoanManager Classes\n");

      // Instantiate lm as instance of LoanManager class
      LoanManager lm = new LoanManager();

      // Call createLoan method of LoanManager instance to
      // create and return new Loan instance
      Loan ln = lm.createLoan("House", 25, 125000);

      // Print out the information of newly created loan
      System.out.println("Newly created loan info.");
      lm.displayInfo(ln);
      System.out.println("----------------------");

      // Update loan information (from House to Car loan)
      // by using updateLoan method of LoanManager instance
      lm.updateLoan(ln, "Car", 7, 56000);
      System.out.println("Updated loan info.");
```

```
        lm.displayInfo(ln);
    }
}
```

The `TestLoan.java` program produces an output as follows:

```
Test Loan and LoanManager Classes

Newly created loan info.
Type: House
Years: 25
Amount: 125000.0
Yearly interest: 9000.0
----------------------
Updated loan info.
Type: Car
Years: 7
Amount: 56000.0
Yearly interest: 4032.0
```

Complete the implementation of `LoanManager` class below, so it can be used to create and handle the `Loan` instance as demonstrated in the `TestLoan.java` program above.

(5 marks)

```
//Class to manage Loan instance
class LoanManager {

   //(a)Method to create and return new Loan instance
   public ____ createLoan (____, _____, _____) {
      Loan l = ___________________;
      ___________________;
   }

   //(b)Method to update Loan instance
   public ____ __________ (_____, _____, _____, _____) {
      ________________;
      ________________;
      ________________;
   }

   // Method to display Loan instance info.
   public void displayInfo(Loan ln) {
      System.out.println("Type: " + ln.getType());
      System.out.println("Years: " + ln.getYears());
      System.out.println("Amount: " + ln.getAmount());
      System.out.println("Yearly interest: " +
                         ln.getAmount() * Loan.INTEREST/100);
   }
}
```

1.3 Draw an UML class diagram to represent `LoanManager` class.

(2 marks)

Question 2 (20 Marks)

2.1 Consider the following Java program:

```
//Program 1
public enum Color {
   RED("FF0000"), GREEN("00FF00"), BLUE("0000FF");

   private String hexCode;

   private Color(String hexCode) {
      this.hexCode = hexCode;
   }

   public String getHexCode() {
      return hexCode;
   }
   public static void main(String [] a) {
      for (Color color : Color.values())
      System.out.println((color.ordinal() + 1) + ") " + color +
                         " has code " + color.getHexCode());
   }
}

```

Based on the program above that defines an enumerated type called **`Color`**, answer the following questions:

a) What is the significance of the values "FF0000", "00FF00", and "0000FF" that are passed as arguments to the constructor of each value in the **`Color`** enumerated type?

(1 mark)

b) What is the purpose of the **`getHexCode()`** method in the **`Color`** enumerated type?

(1 mark)

c) What will be the output of the program?

(3 marks)

d) Write a Java statement in **`main()`** method that uses the **`compareTo()`** method to display the results of a comparison of the positions of **`RED`** and **`BLUE`** in the **`Color`** enumerated type. What will the output be?

(2 marks)

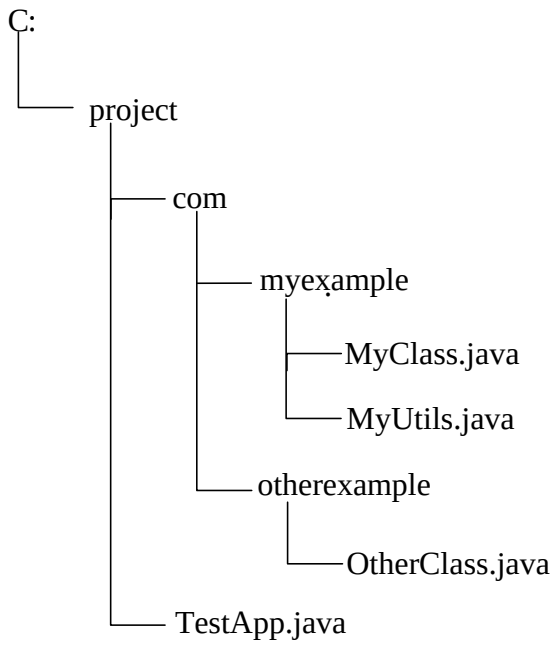
e) Consider adding the following statements in the **`main()`** method:

```
if (Color.GREEN.equals(Color.RED))
    System.out.println("GREEN and RED are two different colors.");
else
    System.out.println("GREEN and RED are the same color.");
```

The statements, however, include an error. What is the error, and how can it be fixed?

(2 marks)

2.2 Given the directory structure as shown below:



a) What is the package name of **`MyClass.java`** and **`MyUtils.java`**?

(1 mark)

b) What is the package name of **`OtherClass.java`**?

(1 mark)

c) In **`TestApp.java`**, how would you import all the classes in **`com.myexample`**?

(1 mark)

d) Suppose you want to move **`MyUtils.java`** to a new package called **`com.myexample.util`**. What changes would you need to make to the directory structure and the code?

(3 marks)

2.3 Consider the Java program below and answer the following questions:

```
//Program 2
public class WrapperClass {
   public static void main(String[] args) {
      Integer num1 = 10;
      Double num2 = 3.14;

      //(a)Converting an object of a wrapper type to
      //its corresponding primitive value
      int value1 = ________________________________;
      double value2 = ______________________________;

      System.out.println("value1: " + value1);
      System.out.println("value2: " + value2);

      //(b)Adding value1 and value2 and storing the
      //result in the result variable by converting
      //it to the Double wrapper class
      Double result = ____________________________;

      System.out.println("result: " + result);
   }
}
```

a) Fill in the blanks with an appropriate Java statement to convert an object of a wrapper type to its corresponding primitive value. **Note:** you must use a method provided in the wrapper class. What is the name of the conversion?

(3 marks)

b) Fill in the blanks with an appropriate Java statement to add **`value1`** and **`value2`** and store the result in the **`result`** variable by converting it to the **`Double`** wrapper class. **Note:** you must use a method provided in the wrapper class.

(2 marks)

Question 3 (30 Marks)

3.1 Consider the following `Compute.java` program:

```
01 // Compute.java
02 public class Compute {
03     public static void main(String[] args) {
04         int[] nums = { 4, 3, 2, 1 };
05
06         for (int i = 1; i < nums.length; i++)
07             if (nums[i - 1] < nums[i]) {
08                 int t = nums[i];
09                 nums[i] = nums[i - 1];
10                 nums[i - 1] = t;
11             }
12
13         for (int i = 0; i < nums.length; i++) {
14             System.out.print(nums[i] + " ");
15         }
16
17         System.out.println();
18     }
19 }
```

a) Complete the table below by tracing the output of the `Compute.java` program for different integer item values assigned to the `nums` array parameter.

(6 marks)

nums Parameter Values	**Output**
{ 4, 3, 2, 1 }	4 3 2 1
{ 1, 3, 5, 7 }	
{ 6, 5, 4, 3 }	
{ 3, 6, 2, 5 }	

b) Assume that you want to modify the `Compute.java` program based on the following new requirements:

- **Req. #1:** specifically define `nums` array parameter to store 4 integer numbers
- **Req. #2:** create the `Scanner` class instance namely `scan` to get input from `System.in` input stream
- **Req. #3:** iterate and use the `nextInt()` method of the `scan` instance to input values into the `nums` array parameter

Write the Java code for each requirement listed above.

(5 marks)

3.2 Given the example of the input/output of the `Array2DMax.java` program as follows:

```
Enter the number of rows in the matrix: 2
Enter the number of columns in the matrix: 3
Enter the values for the matrix, row by row:
Row 0:
1
2
3
Row 1:
4
5
6
The sum of row 0 is 6
The sum of row 1 is 15
The largest element is 6
It is located at row 1 and column 2
```

Guided by the above input/output example, complete the implementation of the `Array2DMax.java` program below. Answer this question by filling in the blank line sections placed after the comments started from **(a)** to **(f).**

(9 marks)

```
01 // Array2DMax.java
02 import java.util.Scanner;
03
04 public class Array2DMax {
05    public static void main(String[] args) {
06       Scanner input = new Scanner(System.in);
07
08       System.out.print("Enter the number of rows in the matrix: ");
09       int numRows = input.nextInt();
10
11       System.out.print("Enter the number of columns in the matrix: ");
12       int numCols = input.nextInt();
13
14       //(a)Create empty 2D array of integer named "matrix" with
15       //specified dimensions [2 marks]
16       ______________________________;
17
18       System.out.println("Enter the values for the matrix, row by row:");
19
20       for (int i = 0; i < numRows; i++) {
21          System.out.println("Row " + i + ": ");
22
23          for (int j = 0; j < numCols; j++) {
24             //(b)Populate the matrix with user input [2 marks]
25             ______________________________;
26          }
27       }
28
29       //(c)Set first element as the largest element (maxVal) [1 mark]
30       int maxVal = _____________;
31
32       int maxRow = 0; // row index of largest element
```

```
33        int maxCol = 0; // column index of largest element
34
35        for (int i = 0; i < numRows; i++) {
36           int rowSum = 0; // sum of current row's elements
37           for (int j = 0; j < numCols; j++) {
38
39              //(d)Compute row sums [1 mark]
40              ______________________;
41
42              if (matrix[i][j] > maxVal) {
43
44                 //(e)Update largest element [1 mark]
45                 maxVal = _____________;
46
47                 //(f)Update row index & column index of
48                 //largest element [2 marks]
49                 maxRow = ____;
50                 maxCol = ____;
51              }
52           }
53
54           System.out.println("The sum of row " + i + " is " + rowSum);
55           input.close();
56        }
57
58        System.out.println("The largest element is " + maxVal);
59        System.out.println("It is located at row " + maxRow +
60                           " and column " + maxCol);
61     }
   }
```

3.3 Consider the `Fruit.java` class below:

```
01 // Fruit.java
02 Public class Fruit {
03     private String name;
04     private double sugarLevel; // grams sugar / 100 grams fruit
05
06     public Fruit(String n, double sl) {
07         name = n;
08         sugarLevel = sl;
09     }
10
11     public String getName() {
12         return name;
13     }
14
15     public double getSugarLevel() {
16         return sugarLevel;
17     }
18 }
```

Table 1 below presents fruit sugar level information (grams sugar/100 grams fruit) to be handled by `Fruit` class.

Fruit	**Sugar Level**
Apple	13.3
Guava	6
Mango	14.8

Table 1: Fruit sugar level

Given the output of `TestFruit.java` program as follows:

```
Fruit Sugar Level Info.:
Apple - 13.3
Guava - 6.0
Mango - 14.8
Fruit with lowest sugar level is Guava
```

Complete the `TestFruit.java` program below to produce output as shown above. Answer this question by filling in the blank line sections placed after the comments started from **(a)** to **(g).**

(10 Marks)

```
01 public class TestFruit {
02    public static void main(String[] args) {
03       //(a)Declare fruits as an array of Fruit objects
04       _______ fruits = ____________; // [1 mark]
05
06       //(b)Create and assign Fruit instances to the fruits array
07       _____________________________________; // [1.5 marks]
08       _____________________________________; // [1.5 marks]
09       _____________________________________; // [1.5 marks]
10
11       System.out.println("Fruit Sugar Level Info.:");
12       for (Fruit fr : fruits) {
13          //(c)Print out fruit's sugar level information
14          System.out.println(_________________ + " - " +
15                             _________________); // [1 mark]
16       }
17
18       // Find and print out fruit with lowest sugar level
19       Fruit fruitLowest = lowestSugar(fruits);
20       System.out.println("Fruit with lowest sugar level is " +
21                          fruitLowest.getName());
22    }
23
24    // Method to find and return fruit with lowest sugar level
25    public static Fruit lowestSugar(Fruit[] fList) {
26       //(d)Set the first item in the fruit array
27       //as fruit with the lowest sugar level
28       Fruit lowest = _________; // [0.5 marks]
29
```

```
30        for (int i = 0; i < fList.length; i++) {
31
32           //(e)Check if need to update the fruit with
33           //lowest sugar level
34           if (_________________ > ____________________) { // [1.5 marks]
35
36              //(f)Update lowest sugar level fruit
37              _________________; // [1 mark]
38           }
19        }
20
21        //(g)Return fruit with lowest sugar level
22        _____________; // [0.5 marks]
23     }
24  }
```